

Polymorphism

in C#



Operator Overloading

Mr. Bangar Raju

 **NARESHO**
technologies

Understanding Polymorphism in C#: Method Overriding and Operator Overloading

By Albert Harmon

Published July 31, 2017

🕒 4 min read

Education



TLDR This blog post explores the concepts of method overriding and operator overloading in C#. It provides detailed explanations, examples, and code snippets

to illustrate how these features enhance the functionality and flexibility of C# applications.

In this blog post, we will delve into two important concepts in C#: method overriding and operator overloading. These features are essential for creating flexible and reusable code in object-oriented programming. We will explore their definitions, uses, and practical examples to illustrate how they work in C# applications.

What is Method Overloading?

Method overloading is an approach that allows a class to define multiple behaviors for a method, depending on the parameters passed to it. This means that the same method name can be used with different parameter types or numbers, enabling different functionalities.

Example of Method Overloading

Consider the `Substring` method in the `string` class. This method can be used in two different ways:

1. **Single Parameter:** `string.Substring(int startIndex)` - This retrieves a substring starting from the specified index to the end of the string.
2. **Two Parameters:** `string.Substring(int startIndex, int length)` - This retrieves a substring starting from the specified index for a

specified length.

For example:

```
string str = "Hello, how are you?";  
string result1 = str.Substring(14); // Returns "you?"  
string result2 = str.Substring(10, 3); // Returns "are"
```

In this case, the `Substring` method is overloaded to provide different functionalities based on the parameters.

What is Operator Overloading?

Operator overloading is similar to method overloading but applies to operators. It allows you to define multiple behaviors for an operator based on the operand types. This means that the same operator can perform different operations depending on the types of the operands involved.

Example of Operator Overloading

A common example is the `+` operator. When used with numeric operands, it performs addition, while with string operands, it performs concatenation:

- **Addition:** `int result = 10 + 20; // result is 30`

- **Concatenation:** ``string combined = "Hello" + " World"; // combined is "Hello World"``

The behavior of the ``+`` operator changes based on the types of the operands.

Implementing Operator Overloading in C#

To implement operator overloading in C#, you need to define a special method called an operator method. This method must be static and has a specific naming convention. Here's how you can define an operator method for addition in a custom class:

Example: Adding Two Matrices

Let's create a ``Matrix`` class that represents a 2x2 matrix and overload the ``+`` operator to add two matrices together:

```
public class Matrix
{
    public int A, B, C, D;

    public Matrix(int a, int b, int c, int d)
    {
        A = a;
        B = b;
```

```
        C = c;  
        D = d;  
    }  
  
    public static Matrix operator +(Matrix m1, Matrix m2)  
    {  
        return new Matrix(m1.A + m2.A, m1.B + m2.B, m1.C + m2.  
    }  
}
```

With this implementation, you can now add two `Matrix` objects using the `+` operator:

```
Matrix m1 = new Matrix(1, 2, 3, 4);  
Matrix m2 = new Matrix(5, 6, 7, 8);  
Matrix result = m1 + m2; // result is a new Matrix with values
```



Method Overriding in C#

Method overriding allows a derived class to provide a specific implementation of a method that is already defined in its base class. This is useful when you want to change or extend the behavior of a method inherited from a parent class.

Example of Method Overriding

Consider the `ToString` method, which is inherited from the `Object` class. By default, it returns the name of the class. You can override this method in your `Matrix` class to return a formatted string representation of the matrix:

```
public override string ToString()
{
    return $"[{A}, {B}]\n[{C}, {D}]";
}
```

Now, when you call `ToString()` on a `Matrix` object, it will return a nicely formatted string instead of the class name.

Conclusion

In this blog post, we explored the concepts of method overloading and operator overloading in C#. We learned how these features enhance the flexibility and reusability of code by allowing methods and operators to behave differently based on their parameters or operand types. By implementing these concepts, developers can create more intuitive and powerful applications in C#.

Understanding and utilizing these features is crucial for any C# developer looking to write clean, efficient, and maintainable code.